

PER%SENSE — LEGACY DOS APPLICATION

Persense Financial Math Explainer

A plain-language walkthrough of the financial mathematics behind Per%Sense — building from a single idea up to the complete approach, and explaining why it works.

What this is. A conceptual explainer of the *mathematics* Per%Sense uses to value loans, mortgages, cash-flow streams, and life-contingent payments. It builds the ideas up in layers and stays high-level: there is no program code here, and no dependence on how the software is written.

Source. This document is derived solely from the original **DOS application** — the mathematics as the program was conceived and implemented.

Who it is for. Anyone who wants to understand *what* Per%Sense computes and *why* the answers are correct — analysts, reviewers, and engineers alike. No prior finance background is assumed; the ideas are introduced from first principles.

Contents

- 1 Introduction: one idea, many screens**
- 2 The time value of money**
- 3 Measuring time: day-count conventions**
- 4 The annuity: a geometric series at the core**
- 5 Amortizing a loan**
- 6 The present value of arbitrary cash flows**
- 7 Mortgages and the true cost of borrowing**
- 8 Life contingencies: paying only if...**
- 9 How a calculation actually proceeds**
- 10 The iterative solver in depth**
 - 10.1 The error function · 10.2 Newton & the secant · 10.3 Safeguards · 10.4 Bisection · 10.5 Two unknowns at once
- 11 Rounding, precision, and trust**
- 12 Why it all works**

1 Introduction: one idea, many screens

Per%Sense looks like several different calculators — a mortgage sizer, a loan amortizer, a present-value worksheet, an actuarial annuity tool. Underneath, it is one idea applied with increasing richness.

That idea is the **time value of money**: a dollar promised in the future is worth less than a dollar in hand today, and the exact amount less is governed by an interest rate. Every screen in Per%Sense is, at heart, a way of (a) writing down a set of dated money movements, (b) translating them all to a common point in time so they can be compared and added, and (c) solving for whatever the user left unspecified.

Two themes recur throughout and are worth naming up front:

- **Everything reduces to discounting and summing.** Whether the cash flows are a single balloon payment, a level monthly mortgage payment, an escalating pension, or a benefit paid only on death, the machinery is the same: move each amount through time with an interest factor, then add the pieces. Where the payments are regular, the sum collapses into a single compact formula — a geometric series — which is the mathematical engine the whole program turns on.
- **The user fills in the blanks.** Per%Sense does not have a fixed “inputs here, answer there” flow. The user supplies whatever they know and leaves *one* field blank; the program figures out which quantity is missing and solves for it. Mathematically this is treating each screen as a small system of equations in which any single variable can be made the unknown. Sometimes the algebra rearranges cleanly to give the answer directly; sometimes the unknown is tangled up inside the equation and must be found by successive approximation.

The shape of this document. We start with the single building block (the time value of money), add the geometric series that lets us handle whole streams of payments at once, then apply that engine in turn to loans, to general present-value problems, to mortgages, and finally to life-contingent payments. We close with how the program inverts these relationships to solve for an unknown, and why the results can be trusted.

2 The time value of money

Money has a price for time. If the prevailing interest rate is positive, then having a dollar now is strictly better than being promised a dollar a year from now, because today's dollar can be put to work in the meantime. The reverse operation — asking “what is a future dollar worth to me today?” — is called **discounting**, and it is the operation Per%Sense performs more than any other.

Compounding and the discount factor

If interest accrues and is itself reinvested, money grows by a multiplicative factor over time. Per%Sense works with **continuous compounding**, in which the growth factor over an interval of t years at rate r is the exponential $e^{r \cdot t}$. Discounting is the inverse: a payment of C due t years from now is worth, today,

$$\text{Present value} = C \cdot e^{-r \cdot t}.$$

The factor $e^{-r \cdot t}$ is always between 0 and 1 for a future payment, so discounting shrinks future money — and the further away or the higher the rate, the more it shrinks.

Why continuous compounding? One could compound monthly, quarterly, or annually instead. Per%Sense chooses the continuous convention internally because it makes the algebra clean: factors for adjacent periods simply multiply ($e^{r \cdot t_1} \cdot e^{r \cdot t_2} = e^{r \cdot (t_1 + t_2)}$), time can be measured in fractional years without special cases, and the sums of payment streams telescope neatly. The continuously-compounded rate is sometimes called the *force of interest*; it is the program's internal “true rate.”

Two ways to quote the same rate

People do not usually think in continuous rates. A loan is quoted as, say, “6% compounded monthly,” meaning the nominal annual rate is 6% and interest posts twelve times a year. Per%Sense therefore keeps *two* views of every rate and converts between them at the edges of the calculation:

$$\text{yield from rate: } y = n \cdot (e^{r/n} - 1) \quad \text{rate from yield: } r = n \cdot \ln(1 + y/n)$$

Here r is the internal continuous rate, y the human-facing nominal rate compounded n times a year, and the two formulas are exact inverses of one another. The user enters and reads a yield; the engine computes in the continuous rate. This separation — a clean internal representation, converted to a familiar one only for display — is a deliberate design choice that keeps every internal formula in its simplest form.

WORKED EXAMPLE — DISCOUNTING, AND THE TWO VIEWS OF A RATE

A promise of **\$10,000 in 5 years**, at a 6% continuous rate, is worth today

$$\$10,000 \times e^{-0.06 \times 5} = \$10,000 \times e^{-0.30} = \$10,000 \times 0.7408 = \$7,408.$$

And a rate quoted as **6% compounded monthly** becomes the internal continuous rate

$$r = 12 \times \ln(1 + 0.06/12) = 12 \times \ln(1.005) = 5.985\%,$$

which converts straight back: $12 \times (e^{0.05985/12} - 1) = 6.00\%$. The two views describe the same money; the engine computes in the continuous one and shows the quoted one.

3 Measuring time: day-count conventions

Interest is rate times *time* times principal, so before we can compute anything we must answer a deceptively hard question: **how much of a year is the gap between two dates?** There is no single right answer, because the answer is a market *convention*, and different markets use different ones. Per%Sense supports the common conventions because using the wrong one produces a subtly — and sometimes not so subtly — wrong number.

Convention	How a year is measured	Typical use
30/360	Every month is treated as 30 days and every year as 360, with small corrections at month-ends. The span between two dates is a tidy mix of whole months and 30-day fractions.	Many loans and bonds; gives clean, predictable payment amounts.
Actual / 365–366	Count the real calendar days between the dates and divide by the actual length of the year (365, or 366 in a leap year). Used for genuine day-count loan interest.	Loans where interest accrues on actual days outstanding.
Actual / 365.25	Real days divided by the average year length 365.25, used for the present-value screens.	Valuation problems where a smooth average year is preferred.

The differences look small but compound across a long schedule. A thirty-year mortgage has 360 interest calculations; a fraction-of-a-day discrepancy in each, accumulated, moves the final balance by real money. This is why Per%Sense is careful to count time the same way the relevant market does, and why the choice of basis is a first-class input rather than a hidden default.

Dates themselves are handled by converting each calendar date to a single running day-number (a **serial date**), so that the gap between two dates is just a subtraction, and advancing a payment by a month or a quarter is arithmetic on the calendar with sensible month-end behavior (the end of January advanced one month lands on the end of February, not a non-existent February 30th).

WORKED EXAMPLE — THE SAME SIX MONTHS, TWO CONVENTIONS

From 1 January to 1 July 2024 is exactly half a calendar year — but “half” depends on the rule. Under **30/360** the span is 6 months ÷ 12 = **0.5000 year**. Under an **actual-day** rule it is 182 real days ÷ 366 (a leap year) = **0.4973 year**. On \$100,000 at 8%, one period's interest is therefore **\$4,000.00** the first way and **\$3,978.14** the second — a \$22 gap from the choice of convention alone, which then compounds across a long schedule.

4 The annuity: a geometric series at the core

Discounting a single payment is easy. The reason Per%Sense can value a thirty-year stream of payments instantly is that a *regular* stream collapses into one formula.

Consider a level stream: the same payment A , made at the end of each period, for n periods. Each payment must be discounted back to today, and payments further away are discounted more. If the discount factor for *one* period is v (a number a little less than 1), then the first payment is worth $A \cdot v$, the second $A \cdot v^2$, the third $A \cdot v^3$, and so on. The total present value is

$$PV = A \cdot (v + v^2 + v^3 + \dots + v^n).$$

The sum in parentheses is a **geometric series** — each term is a fixed multiple of the one before — and geometric series have a famous closed form. The whole stream's value compresses to:

$$PV = A \cdot v \cdot (1 - v^n) / (1 - v).$$

This is the **annuity factor**, and it is the single most important formula in Per%Sense. It says: to value a level stream, you do not have to add up hundreds of payments one by one — the regularity lets you replace the whole sum with one expression involving only the per-period discount factor v and the number of payments n .

Why this one formula goes so far. Almost every Per%Sense quantity is some version of this annuity factor. A loan payment is the number that makes the annuity value of the payments equal the amount borrowed. A mortgage price is an annuity relationship between payment and principal. A pension's worth is an annuity value of its payments. Master this formula and you have the spine of the entire program.

The degenerate case, and why it is handled deliberately

When the interest rate is exactly zero, the discount factor v equals 1, and the formula above becomes $0 / 0$ — undefined as written. But the *meaning* is perfectly clear: with no interest, every payment is worth its face value, so a stream of n payments of A is simply worth $n \cdot A$. Per%Sense recognizes when the rate is close enough to zero that the general formula would misbehave, and substitutes the plain count. This is a recurring pattern: the program watches for the handful of inputs where a clean formula degenerates, and supplies the correct limiting value by hand. We will see the same care with escalating payments (Section 6) and near-zero quantities in the numerical routines (Section 10).

WORKED EXAMPLE — VALUING A FIVE-YEAR STREAM

The present value of **\$1,000 a year for 5 years at 8%** uses the annuity factor with one-year discount factor $v = 1/1.08 = 0.9259$:

factor = $v(1 - v^5)/(1 - v) = 3.9927$, so **PV** = $\$1,000 \times 3.9927 = \$3,992.71$.

Five separate discountings replaced by one expression. At a **0% rate** the factor would instead be the plain count, 5, giving exactly **\$5,000** — the degenerate case supplied by hand.

5 Amortizing a loan

An amortizing loan is one repaid by regular payments, each part interest and part principal, until the balance reaches zero. Per%Sense can both *build the schedule* forward and *solve* for any missing piece.

The forward schedule: a simple recurrence

Walk a loan period by period. At the start of a period the borrower owes a balance B . Over the period, interest accrues on that balance — rate times the length of the period times B . Then the payment is applied: it first covers the interest, and whatever is left reduces the principal. The new balance is

$$B_{\text{new}} = B + \text{interest} - \text{payment}, \quad \text{where interest} = B \cdot (\text{rate for one period}).$$

Apply this rule repeatedly and you generate the entire amortization schedule: each row's interest, principal, and remaining balance. Early on, the balance is large, so most of the payment is interest; as the balance falls, more of each payment goes to principal. The schedule is nothing more than this one recurrence iterated.

The level payment: inverting the annuity

What payment exactly retires the loan in n periods? The present value of all the payments, discounted at the loan rate, must equal the amount borrowed. That is precisely the annuity relationship of Section 4, read backwards: given the principal, the rate, and the number of payments, solve the annuity formula for the payment A . Because the principal appears linearly, this inversion is clean algebra — a direct formula, no trial and error. The same level-payment idea, with the annuity factor rearranged, lets the program instead solve for the number of payments (how long until the loan is paid off) or, with a little more work, the rate.

Principled variations on the basic loan

Real loans depart from the textbook case in specific, well-defined ways. Per%Sense treats each departure as a precise modification of the basic recurrence rather than an ad-hoc tweak:

Variation	The idea
Payment in advance	Payments fall at the <i>start</i> of each period rather than the end (an “annuity-due”). Because each payment arrives a period sooner, it is worth slightly more, so the required payment is a touch smaller. The discounting is adjusted by one period's factor throughout.
Odd first period	The gap from closing to the first payment is rarely exactly one period. The first period's interest is <i>prorated</i> by the true length of that stub — a fraction of a normal period — so the schedule begins correctly even when the calendar does not cooperate.
Rule of 78	An older method that front-loads interest: the total finance charge is divided among the periods by sum-of-the-digits weights (the first period gets the largest share, the last the smallest). Per%Sense reproduces this as a linearly declining interest charge whose pieces add up to the same total finance charge.
U.S. Rule	A convention in which unpaid interest is tracked separately and is <i>not</i> itself charged interest — interest never compounds on interest. The program carries a side bucket of uncovered interest and accrues only on principal that has had its interest paid.

Shaping the cash flows: advanced options

Beyond level payments, Per%Sense allows the cash-flow stream to be sculpted: **balloons** (a large one-time payment on a chosen date), **prepayments** (extra periodic amounts over a sub-interval), **rate adjustments** (an adjustable-rate loan whose rate or payment changes on set dates), a **moratorium** (an interest-only deferment period), a **target** (a minimum principal reduction per payment), and **skipped months**. Mathematically none of these change the engine — the period-by-period recurrence simply consults, at each date, whatever events apply: it adds a balloon, suppresses a skipped payment, re-computes the payment after a rate change, and so on. The richness lives in the schedule of events, not in new mathematics.

WORKED EXAMPLE — A CAR LOAN

Borrow **\$25,000 over 5 years (60 monthly payments) at 6%** (0.5% per month). The level payment is the annuity inversion — the payment whose 60 discounted instalments total \$25,000:

$$\text{payment} = \$25,000 \times 0.005 / (1 - 1.005^{-60}) = \$483.32.$$

The first payment splits into **\$125.00 interest** ($\$25,000 \times 0.5\%$) and **\$358.32 principal**, leaving \$24,641.68. Iterating that split, the balance reaches exactly zero at payment 60 — the recurrence and the level payment working together.

6 The present value of arbitrary cash flows

The present-value screen generalizes the loan idea: instead of one borrower and a level payment, value *any* collection of dated amounts against a chosen valuation date.

Two kinds of cash flow are allowed: **single payments** (a lump on a date) and **periodic streams** (a level amount paid so many times a year between two dates). Each single payment is discounted by the time-value factor of Section 2; each stream is valued by the annuity factor of Section 4; and the grand total is just their sum, all expressed as of the common “as-of” date. This is the same discount-and-sum recipe, now over a freely chosen set of flows.

Escalating payments (cost-of-living adjustments)

Pensions and settlements often grow over time. Per%Sense lets a stream carry a cost-of-living rate, so each payment is a little larger than the last. Now two forces act on each payment: it *grows* with the escalation rate and *shrinks* with discounting. Their net effect per period is captured by a single combined factor — the escalation working against the discount — and the stream is again a geometric series, just with this net ratio in place of the plain discount factor. The annuity machinery carries straight over.

When growth equals the discount rate. If a stream grows at exactly the discount rate, the two forces cancel: every payment, escalated and then discounted, is worth precisely the same amount today. The geometric ratio becomes 1 and the stream's value is simply the number of payments times one payment's present value. This is the same degeneracy as the zero-rate annuity (Section 4), and Per%Sense handles it the same deliberate way — recognizing the cancellation and substituting the plain count rather than dividing by zero.

Streams that run forever

A stream with no end date is a **perpetuity**. Such a thing has a finite present value only if discounting outruns growth — if the discount rate exceeds the escalation rate, the ever-more-distant payments shrink fast enough that their infinite sum converges. If growth keeps pace with or outruns discounting, the value is infinite, and Per%Sense says so rather than returning a meaningless number. This is not a quirk; it is the honest mathematical answer, and flagging it is part of getting the math right.

WORKED EXAMPLE — AN ESCALATING PENSION

A pension pays **\$12,000 a year, rising 3% annually, for 20 years**, discounted at 6%. Growth and discounting combine into a single net ratio per year, and the geometric sum gives a present value of about **\$174,800**. Now suppose the pension grew at the full 6% — matching the discount rate. Then every payment, escalated and then discounted, is worth exactly the first payment's value, \$11,321, so the whole stream is simply **$20 \times \$11,321 = \$226,415$** — the growth-equals-discount degeneracy, where the geometric ratio becomes 1 and the sum is just the count.

7 Mortgages and the true cost of borrowing

A mortgage screen is an annuity problem dressed in the vocabulary of home buying — price, down payment, points, monthly payment — plus two questions unique to comparing loans: what does it really cost, and which of two loans is cheaper?

The web of relationships

The quantities on a mortgage are tied together by simple relationships. The amount financed is the price less the down payment. The cash a buyer needs is the down payment plus points charged on the loan. The monthly payment is the annuity payment that amortizes the financed amount over the term — the very inversion of Section 5 — plus a flat allowance for taxes and insurance that is added on but never amortized. Because these relationships are mutually consistent, the user can specify whichever subset they know — price and down payment to get the payment, or a target payment to get the most expensive affordable house — and the program rearranges the same relationships to fill in the rest. A balloon, if present, is just the present value of one extra future lump folded into the same equation.

APR: the true cost as an internal rate of return

The quoted rate is not the true cost of a loan, because the borrower pays **points** up front and so receives less cash than the face amount. The **annual percentage rate** answers: what single rate, applied to the money actually advanced, reproduces the stream of payments the borrower makes? Formally, it is the discount rate at which

$$(\text{present value of all payments and any balloon}) = (\text{amount financed}) \times (1 - \text{points}).$$

The right-hand side is the net cash the lender actually put out after withholding points; the left-hand side discounts everything the lender gets back. The APR is the rate that makes them equal — an **internal rate of return**. Because points reduce the cash advanced but not the payments, the APR is always at least the note rate, and more so for a shorter loan over which the up-front points are spread thinner. This rate cannot be written down in closed form — it sits inside the discounting — so it is found by the successive-approximation methods of Section 9.

The crossover: which loan is cheaper, and when

A classic dilemma: a low-rate loan with high points versus a high-rate loan with low points. The high-points loan only pays off if you keep it long enough to amortize those points. Per%Sense computes the **crossover** — the holding period at which the two loans' effective costs are exactly equal. Below it, one loan is cheaper; above it, the other. Mathematically this is finding the point at which two cost curves intersect: a pair of equations (each loan's value equals its net advance) solved simultaneously for the crossover rate and time. There is one subtlety the program handles: if a loan has a balloon, its cost curve has a sudden step at the balloon date, and the crossover may land exactly there — so the program checks the balloon dates directly when the smooth search stalls.

WORKED EXAMPLE — SIZING A MORTGAGE AND ITS TRUE COST

On a **\$200,000 home, 20% down, 2 points, 30 years at 6%**: the amount financed is \$160,000; the cash needed is the \$40,000 down payment plus \$3,200 of points = **\$43,200**; and the monthly principal-and-interest payment is the annuity payment for \$160,000 at 0.5% per month over 360 months = **\$959.28**. Because the 2 points mean the borrower effectively receives only \$156,800, the true cost — the rate at which those 360 payments discount back to \$156,800 — works out to an **APR of about 6.24%**, a little above the 6% note rate. That last figure has no closed form; it is found by the iterative solver of Section 10.

8 Life contingencies: paying only if...

The most sophisticated screen values payments that happen only under a condition involving human life — an annuity that continues only while someone is alive, or a benefit paid when someone dies. The mathematics is a graceful extension of everything so far.

Expected present value

If a payment will be made only if a certain person is still alive on its date, then from today's vantage point the payment is uncertain. The right way to value an uncertain payment is its **expected** present value: the ordinary present value, multiplied by the *probability* that the payment actually happens. So the whole life-contingent valuation is the familiar discount-and-sum of the earlier sections, with one extra factor attached to each cash flow — the probability that its condition is met.

$$\text{value of a life-contingent payment} = (\text{amount}) \times (\text{discount factor}) \times (\text{probability the condition holds}).$$

Survival probabilities from a mortality table

Those probabilities come from a **mortality table**, which records, for a large starting population, how many survive to each age. The chance that a person now aged x is still alive t years later is simply the ratio of survivors at age $x+t$ to survivors at age x :

$$\text{probability of surviving } t \text{ more years} = (\text{survivors at age } x+t) / (\text{survivors at age } x).$$

Because ages fall between the whole-year entries of the table, the program interpolates smoothly between them. Every life-contingent payment is weighted by the appropriate survival probability for its date, and the annuity becomes a **life annuity** — valued payment by payment, since the survival weight differs at every date and so cannot be folded into a single closed form.

Two lives, and benefits on death

With two people, the condition can be richer: pays while *both* are alive (joint life), while *either* is alive (last survivor), or while exactly one specific person survives. Assuming the two lives are independent, these combine by elementary probability: both-alive is the product of the two survival probabilities; either-alive is one minus the product of the two death probabilities; and so on. A **payment on death** is valued by summing, over each possible year of death, the benefit times the probability of dying in that year times the discount factor to that time — an expectation taken over *when* death occurs rather than *whether* a survivor is present. In every case the principle is unchanged: discount each possible cash flow, weight it by the probability it occurs, and add.

WORKED EXAMPLE — A LIFE ANNUITY (ILLUSTRATIVE SURVIVAL)

Value **\$10,000 a year while a 70-year-old survives**, discounted at 5%. Each year's payment is worth its discounted value times the chance of being alive to collect it. Using illustrative survival probabilities:

Year	Discount factor	Survival probability	Expected value
1	0.9512	0.97	\$9,227
2	0.9048	0.93	\$8,415
3	0.8607	0.89	\$7,660

4	0.8187	0.84	\$6,877
5	0.7788	0.79	\$6,153

Over these first five years the life annuity is worth **\$38,332**, against **\$43,142** for a certain annuity that pays regardless — the roughly \$4,800 difference is the mortality discount, the value of the payments that may never fall due. (The survival figures here are illustrative; the principle is exact.)

9 How a calculation actually proceeds

We have the ingredients. Now follow a single calculation from the moment the user presses “compute” to the answer, so the mechanics are concrete rather than abstract.

Step one: read the screen by counting

Every row on a screen has a small number of defining quantities — a single payment is defined by its date, its amount, and its present value; a periodic stream by its two dates, its amount, and its value; the screen as a whole by a discount rate and an as-of date. Per%Sense begins by *counting*, on each row, how many of these are supplied and how many are blank. That count classifies the row:

What the count shows	What it means	What the program does
Exactly the right number present	Fully specified — the row can be computed.	Compute it <i>forward</i> : discount and add.
Missing exactly one	Contains an unknown — one quantity to find.	Solve it <i>backward</i> against a target.
Too few present	Under-determined — not enough to do anything.	Ask for more input.
More than enough, and they conflict	Over-determined — the inputs may disagree.	Warn; do not silently pick one.

This counting is the mathematical content of the “fill in the blanks” philosophy: a row that contains exactly one unknown is one equation in one unknown, which is solvable; zero unknowns is something to evaluate; two or more is ambiguous. The program never needs to be told which field is the answer — it deduces it from which field is empty.

Step two: choose a direction

From the per-row classification the program decides the whole screen's direction. If the rate and as-of date are known and every active row is fully specified, the screen runs **forward**: each cash flow is discounted to the as-of date and the results are summed. If instead exactly one quantity across the screen is blank and there is a target value to hit (a desired total, or a row's own stated value), the screen runs **backward**: that one quantity is the unknown to solve for. If two different quantities are blank, there is no unique answer, and the program says so rather than guessing.

Step three: invert directly, or set up a search

Once the single unknown is identified, the program asks one more question: *can this unknown be isolated by algebra?* If the unknown enters its equation **linearly** — a loan amount, a level payment, a single lump's value — then the relationship can be rearranged to put the unknown alone on one side, and the answer is computed in a single exact step. If the unknown is **tangled** — a rate buried inside the discount exponentials, a date that changes both the number of payments and how far each is discounted, a term that changes the count itself — then no rearrangement frees it, and the program builds an *error function* and hands it to the iterative solver of Section 10.

A forward calculation, worked

Take a \$10,000 loan at 1% per month for 12 monthly payments. The level payment is the annuity inversion of Section 5: the payment whose 12-fold discounted stream equals \$10,000. Numerically that is about **\$888.49**. The schedule is then the one-line recurrence iterated:

Month	Interest = balance $\times$ 1%	Principal = payment – interest	New balance
1	\$100.00	\$788.49	\$9,211.51
2	\$92.12	\$796.37	\$8,415.14
...	...	...	...
12	\$8.80	\$879.69	\$0.00

Notice the interest share falling and the principal share rising as the balance declines — and that the twelfth payment lands the balance exactly on zero, because the payment was chosen by the annuity inversion to do precisely that. Forward calculation is nothing more than this: a payment from a formula, then a recurrence.

A backward calculation, worked

Now suppose the user instead supplies the \$888.49 payment and the \$10,000 amount but leaves the *rate* blank. The rate is tangled — it sits inside every discount factor — so the program cannot rearrange for it. Instead it defines a quantity that measures how wrong any guessed rate is, and adjusts the guess until that quantity is zero. That is the iterative solver, and it is worth seeing in full.

10 The iterative solver in depth

When the unknown cannot be isolated, Per%Sense finds it by successive approximation. The method is the same across every screen, and understanding it is understanding the engine that powers every “solve for the rate / date / term”.

10.1 The error function: turning a question into a root

The solver never works on the unknown directly. It works on an **error function** — a single number, depending on a guess x , that is positive when the guess is too far one way, negative when it is too far the other, and exactly zero when the guess is right. Call it $g(x)$. Solving the screen becomes the universal mathematical problem of **finding the root** of g — the value x^* where $g(x^*) = 0$.

The art is choosing g so its root *is* the answer and so it has a clean single sign-change. Per%Sense uses two natural choices:

- **Leftover-balance error.** For a loan whose payment, rate, or amount is unknown, plug the guess in, run the entire amortization schedule, and read the balance remaining at the end. A correct guess retires the loan exactly, so the leftover is zero. Guess a payment too small and the loan is not paid off (positive leftover); too large and it overpays (negative). The error is the terminal balance.
- **Value-gap error.** For a rate, a date, or a term on a present-value or mortgage screen, compute the present value the guess produces and subtract the target it must hit. The error is (value at the guess) – (target value); it is zero exactly when the guess reproduces the target.

Both errors are chosen so that they move steadily in one direction as the guess changes — a higher rate lowers a present value, a larger payment lowers a leftover balance — which guarantees a single crossing and makes the root unambiguous.

10.2 Newton's method and the secant variant

Given an error function, how do we drive it to zero efficiently? **Newton's method** is the classic answer. Pretend, near the current guess, that g is a straight line — its tangent. A straight line's root is trivial to find, so jump to where the tangent crosses zero:

$$x_{\text{next}} = x - g(x) / g'(x),$$

where g' is the slope of the error at the current guess. When g is smooth and the guess is reasonable, this is astonishingly fast — the number of correct digits roughly *doubles* each step, so a few iterations reach the cent.

Why Per%Sense uses the secant form. Newton needs the slope g' , but Per%Sense's error functions have no tidy slope formula — “run the whole schedule and read the leftover balance” is a simulation, not an expression you can differentiate on paper. So the program *estimates* the slope from the two most recent evaluations — the chord through them instead of the tangent:

$$\text{slope} \approx (g(x_{\text{now}}) - g(x_{\text{prev}})) / (x_{\text{now}} - x_{\text{prev}}),$$

giving the update

$$x_{\text{next}} = x_{\text{now}} - g(x_{\text{now}}) \cdot (x_{\text{now}} - x_{\text{prev}}) / (g(x_{\text{now}}) - g(x_{\text{prev}})).$$

This is the **secant method** — Newton with a chord. It needs no derivative, treats the error function as a black box (exactly right when that function is “simulate the schedule”), and converges almost as fast as true Newton.

A worked illustration. With the \$10,000 / \$888.49 / 12-payment loan of Section 9, let the error be the annuity value at a guessed monthly rate, minus the \$10,000 principal. Guess 0.8% and the value comes out about \$128 *too high* (the rate is too low); guess 1.2% and it comes out about \$126 *too low*. The chord through those two points crosses zero at a rate of about **1.00%** — which is exactly the loan’s rate, found in a single secant step from two evaluations. Real cases take a few more steps and tighter guesses, but the mechanism is precisely this.

That single step recovered a rate we already knew. The real power shows when the answer is genuinely unknown and the search must take several steps — so let us solve, in full, for a quantity with no closed form: the true cost of a loan with points.

WORKED EXAMPLE — SOLVING FOR AN APR, STEP BY STEP

Take the loan of Section 9 — payment \$888.49 a month for 12 months — but now charge **2 points**, so the borrower actually receives **\$10,000 × (1 − 0.02) = \$9,800**. The true monthly rate is the one at which the twelve payments discount back to \$9,800. The error function is

$$g(r) = (\text{value of the payments at rate } r) - \$9,800,$$

which we drive to zero. Starting from two nearby guesses, the secant iteration runs:

Step	Guess (monthly rate)	Value of the payments	Error g
0 (start)	1.000%	\$10,000.0	+\$200.0
1 (start)	1.200%	\$9,873.8	+\$73.8
2 (secant)	1.317%	\$9,802.9	+\$2.9
3 (secant)	1.322%	\$9,799.9	−\$0.1

Each new guess is the point where the straight line through the two previous (rate, error) points crosses zero. The error collapses **+\$200 → +\$74 → +\$3 → −\$0.1** — settling on a monthly rate of about **1.322%, an APR of roughly 15.9%**, well above the 12% note rate because two points on a one-year loan are costly. Four evaluations reach the cent.

Why this converges — and why it must. Two properties of the error function guarantee success. First, it is **monotonic**: raising the rate lowers the value of a fixed set of future payments, so g decreases steadily as the rate rises and crosses zero exactly once — there is a single answer, and no false one to be trapped by. Second, it is **gently curved**: over a small interval the value-versus-rate relationship is very nearly a straight line, so the chord the secant draws through two recent points is an excellent stand-in for the true curve, and the place where that chord hits zero lands close to the real root. Each step then uses two fresher, closer points, so the straight-line stand-in grows ever more accurate and the remaining error shrinks faster and faster — the hallmark of the method, visible above as the error falls by a factor of three, then twenty-five, then thirty. The safeguards of the next section ensure that even when a particular problem is less well-behaved, the search still ends with the right answer or an honest report that it could not be found.

10.3 Safeguards: making the search robust

A bare secant iteration can misbehave — a poor slope estimate can fling the next guess somewhere absurd. Per%Sense wraps the iteration in safeguards that turn a fast-but-fragile method into a dependable one:

- **A smart first guess.** The search starts near the answer, not at random: an APR is seeded with the note rate bumped up for amortized points; a loan rate with the payment-to-principal ratio. A good seed cuts the iterations needed and avoids wandering.
- **Step damping.** Each jump is capped, so one bad slope estimate cannot throw the guess to an impossible value — rate steps are limited to a small band, date steps to a bounded number of years. The search is allowed to move quickly but not recklessly.
- **Best-so-far.** The program remembers the guess that produced the smallest error seen so far and is prepared to return *that* rather than wherever the final step happened to land — a guard against an iteration that improves and then overshoots.
- **Restart.** If the search stalls, it restarts from a different seed (the rate solve, for instance, restarts from zero) before giving up.
- **A divergence brake.** If the steps stop shrinking — a sign the method is not homing in — the program accelerates toward termination so a hopeless case fails quickly and cleanly rather than spinning.

Knowing when to stop. The iteration ends on whichever of three conditions comes first: the *step* has become smaller than a tolerance (the answer has settled and further steps would only jiggle the last digits); the *error itself* is within an acceptable band (a loan balance within half a cent of zero, a date within a day of the target); or a maximum number of iterations has been reached. The last is a safety valve: if the search has not converged by then, Per%Sense reports non-convergence honestly rather than returning a number from a search that never settled.

10.4 When the fast method fails: bisection

Newton and the secant assume the error function is smooth — that a tangent or chord is meaningful. Some of Per%Sense's errors are *not* smooth. The amortization schedule, in particular, “forces” the final payment to clear the balance exactly and then stops; as a guess changes, the number of payments can flip, and the leftover-balance error **jumps** at those flips — it is a step function, not a smooth curve. A tangent drawn across a jump points nowhere useful, and Newton can thrash.

For these, Per%Sense falls back to **bisection**, the most robust root-finder there is. Find a guess that is too low (error one sign) and one that is too high (error the other sign), so the root must lie between them. Then repeatedly take the midpoint, check its sign, and discard whichever half no longer brackets the root. Each step *halves* the remaining uncertainty, so the answer is squeezed down steadily and inexorably. Bisection cannot be fooled by a discontinuity — it only ever asks “is the root above or below this midpoint?” — and it cannot diverge. The price is speed: it gains a fixed amount of precision per step rather than doubling, so it is reserved for exactly the rough cases where the fast method would misbehave. Choosing the fast method where the error is smooth and bisection where it is not is the central judgement of Per%Sense's solver.

10.5 Two unknowns at once: the crossover

The mortgage crossover (Section 7) must solve for *two* unknowns at once — a rate and a holding time — satisfying *two* equations simultaneously (each loan's value equals its net advance). Newton's idea generalizes cleanly. Estimate how each of the two errors responds to a small nudge in each of the two unknowns — a little two-by-two

table of sensitivities. That table is a local linear model of the whole system, and solving it (a tiny linear-algebra step) gives the joint adjustment that drives *both* errors toward zero together. Repeat until both are within tolerance. The only wrinkle is the familiar one: a balloon makes a loan's value jump at the balloon date, so when the smooth two-dimensional search stalls, the program checks the balloon dates explicitly — the same “step function” care as in the one-dimensional case.

Some inversions are forbidden — and the solver is the reason we can tell. Occasionally there is simply no well-posed root to find. Asking for the *date* of a life-contingent payment from its value is the clearest case: the survival probability that weights the payment depends on the very date we are solving for, so the error function does not move cleanly in one direction — it has no single sign-change to bracket. Per%Sense recognizes such cases and declines, rather than returning a number a confused search happened to stop at. Knowing when a problem is not solvable is as much a part of the mathematics as solving the ones that are.

11 Rounding, precision, and trust

Financial answers must be reproducible to the cent, which raises two mathematical concerns that Per%Sense treats with care.

Rounding is a definition, not an afterthought. Money is carried to the cent, and the rule for breaking a tie at exactly half a cent must be fixed and applied consistently or two correct calculations can disagree by a penny. Per%Sense uses a specific, consistent rule (a tie rounds *down*, toward zero), and applies it at the same points every time. The point is not which rule — it is that the rule is definite and uniform, so the results are auditable.

Precision near the edges. Several of the program's formulas involve quantities that, for certain inputs, become very small differences of nearly-equal numbers — exactly the situation where naive computation loses accuracy. The classic example is the annuity factor when the rate is near zero, or a logarithm of a number near one. Per%Sense substitutes mathematically equivalent expressions (short series approximations) in these neighborhoods, so the answer stays accurate rather than dissolving into rounding noise. This is the same instinct as the zero-rate and growth-equals-rate substitutions of earlier sections: identify where a clean formula misbehaves and supply the correct limiting behavior by hand.

Together these make the results **deterministic and auditable**: the same inputs always produce the same figures, and every figure can be traced to a defined calculation — a non-negotiable property for software that values money.

12 Why it all works

Step back and the whole edifice rests on one idea, extended in layers.

At the base is the time value of money: a future amount is worth its discounted value today, and discounting is multiplication by an exponential factor. Add the observation that a *regular* stream of payments, each discounted, forms a geometric series — and the entire valuation of any level stream collapses into a single annuity formula. That formula is the engine. A loan's payment is the annuity inversion; its schedule is a one-line recurrence iterated; a mortgage's relationships are annuity relationships; a present-value worksheet is annuities and lumps summed against a common date; an escalating pension is an annuity with a modified ratio; a life annuity is the same sum with each term weighted by a survival probability. Nothing genuinely new is introduced as the problems grow richer — only more elaborate cash-flow shapes fed through the same discount-and-sum machinery.

On top of that machinery sits the inverse problem — letting the user solve for any one quantity — handled by clean algebra where the unknown is linear and by reliable numerical search where it is not. And wrapping everything is a discipline of precision: deliberate handling of the degenerate cases, a fixed rounding rule, and accurate computation near the numerical edges, so the answers are trustworthy to the cent.

That is why Per%Sense works. It is not a collection of separate financial formulas memorized for separate screens. It is one principle — discount future money and add it up — expressed with enough mathematical care that it can be applied to a level loan, an adjustable mortgage with a balloon, an escalating settlement, or a joint-life pension, and give, in every case, the right answer for the right reason.